Ch 8.2: Ensemble Tree Models

Lecture 22 - CMSE 381

Prof. Elizabeth Munch

Michigan State University
::
Dept of Computational Mathematics, Science & Engineering

Wed, Mar 30, 2022

Announcements

Last time:

- 8.1 Decision Trees

This lecture:

- Bagging
- Random forest
- Boosting
- BART

Announcements:

- Mon 3/28: 8.1
- Weds 3/30: 8.2
- Fri 4/1: Lab: 8.3
  - HW #8 Due Friday
- Next week: No Class

Section 1

Last time

First decision tree example

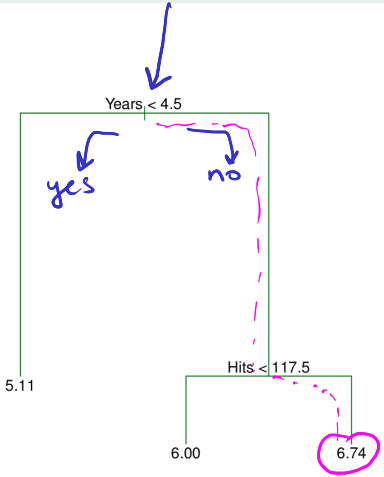
	Hits	Years	LogSalary
1	81	14	6.163315
2	130	3	6.173786
3	141	11	6.214608
4	87	2	4.516339
5	169	11	6.620073
...	...	...	...
317	127	5	6.551080
318	136	12	6.774224
319	126	6	5.953243
320	144	8	6.866933
321	170	11	6.907755

Y

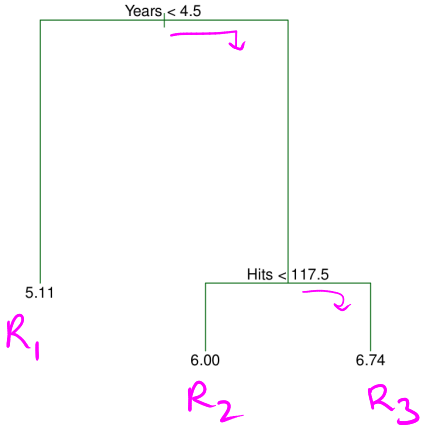
x_i y_i

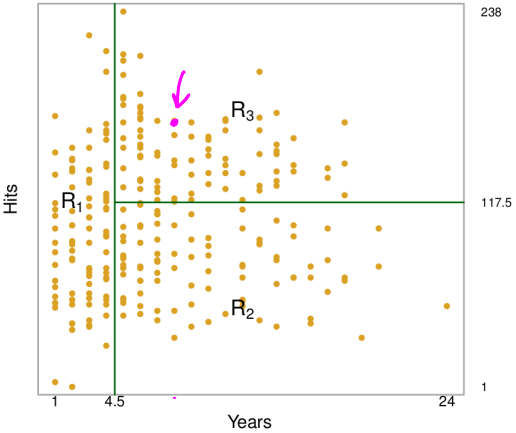
$\hat{y}_i = \hat{y}_{R_j}$

$e^{6.74} \rightarrow \$845{,}346$



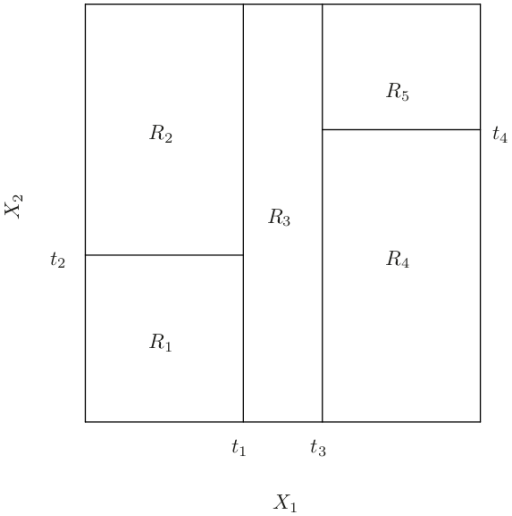
Viewing Regions Defined by Tree





How do we actually get the tree? Two steps

1. We divide the predictor space – that is, the set of possible values for $X_1, X_2, \cdots, X_p$ — into J distinct and non-overlapping regions, $R_1, R_2, \cdots, R_J$.
2. For every observation that falls into the region R_j, we make the same prediction = the mean of the response values for the training observations in R_j.



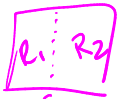
Recursive binary splitting

Goal:
Find boxes $R_1, \cdots, R_J$ that minimize

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

$\hat{y}_{R_j}$ = mean response for training observations in jth box

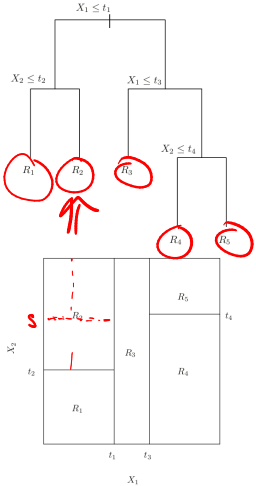
Pick s so that splitting into $\{X \mid X_j < s\}$ and $\{X \mid X_j \geq s\}$ results in largest possible reduction in RSS:



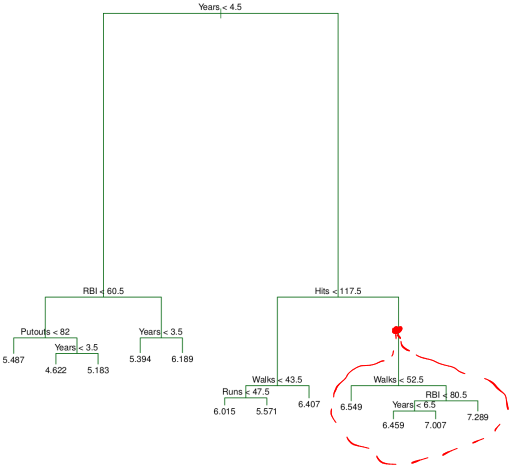
$$R_1(j,s) = \{X \mid X_j < s\}$$
$$R_2(j,s) = \{X \mid X_j \geq s\}$$

x_j

$$\sum_{i \mid x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i \mid x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2$$



Pruning



Remove "small" branches to simplify the tree

Weakest Link Pruning

Also called Cost complexity pruning

For every α, there is a subtree T that minimizes:

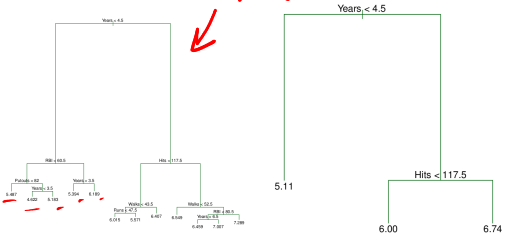
tuning parameter

leaves

$$\sum_{m=1}^{|T|} \sum_{i|x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

- $|T|$ = number of terminal nodes of T
- R_m is rectangle for mth terminal node
- $\hat{y}_{R_m}$ is mean of training observations in R_m

- $\alpha = 0$ gets entire tree
- Increasing α penalizes size of tree
- Branches pruned from tree in nested and predictable fashion
- Easy to get trees for all vales of α
- Pick α via CV



Algorithm version

Algorithm 8.1 *Building a Regression Tree*

1. Use recursive binary splitting to grow a large tree on the training data, stopping only when each terminal node has fewer than some minimum number of observations.

2. Apply cost complexity pruning to the large tree in order to obtain a sequence of best subtrees, as a function of α.

3. Use K-fold cross-validation to choose α. That is, divide the training observations into K folds. For each $k = 1, \ldots, K$:

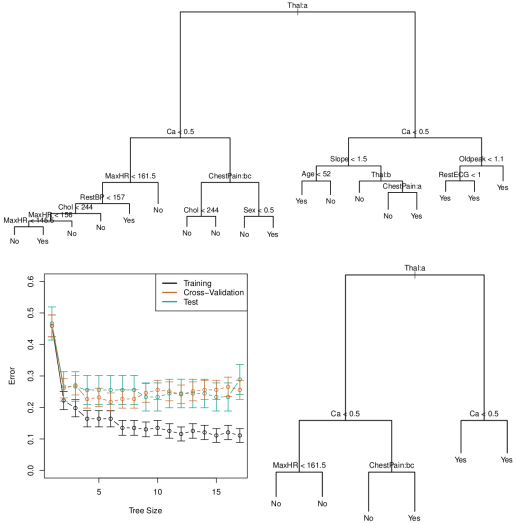
   (a) Repeat Steps 1 and 2 on all but the kth fold of the training data.

   (b) Evaluate the mean squared prediction error on the data in the left-out kth fold, as a function of α.

   Average the results for each value of α, and pick α to minimize the average error.

4. Return the subtree from Step 2 that corresponds to the chosen value of α.

Classification version



Evaluating the splits:

- $\hat{p}_{mk}$ = proportion of training observations in R_m from the kth class
- Error: $E = 1 - \max_k(\hat{p}_{mk})$
- Gini index:

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk})$$

- Entropy:

$$D = -\sum_{k=1}^{K} \hat{p}_{mk} \log \hat{p}_{mk}$$

Section 2

Bagging

Recall: The bootstrap

Idea: Build separate models from independent training sets, and average resulting predictions:

- $\hat{f}^1(x), \cdots, \hat{f}^B(x)$ for B separate training sets
- Return the average

$$\hat{f}_{avg}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^b(x)$$

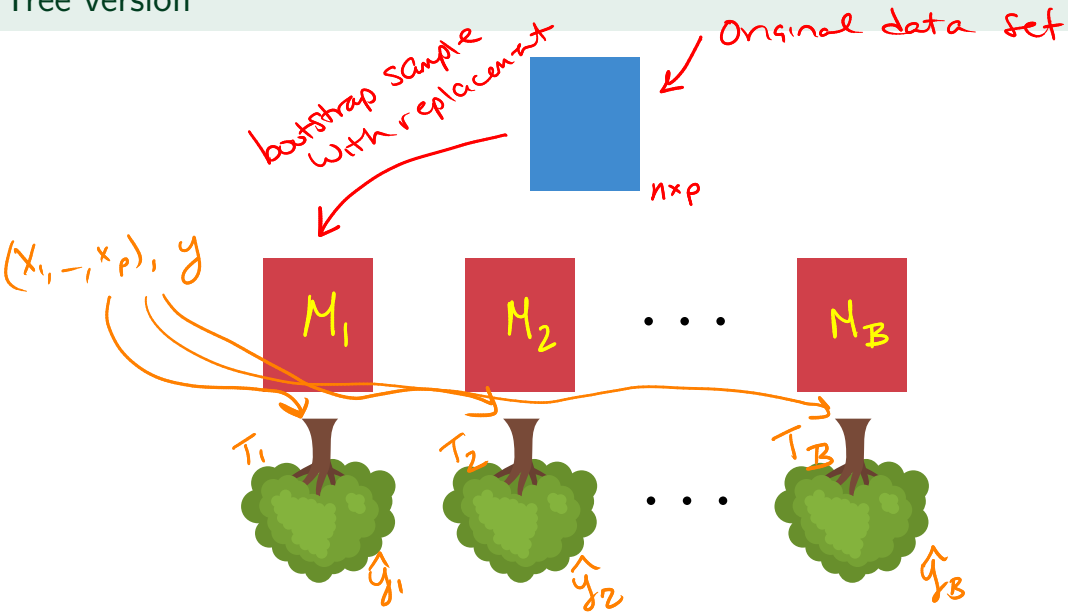
Boostrap modification:

- Work with fixed data set
- Take B samples from this data set (with replacement)
- Train method on bth sample to get $\hat{f}^{*b}(x)$
- Return average of predictions (regression)

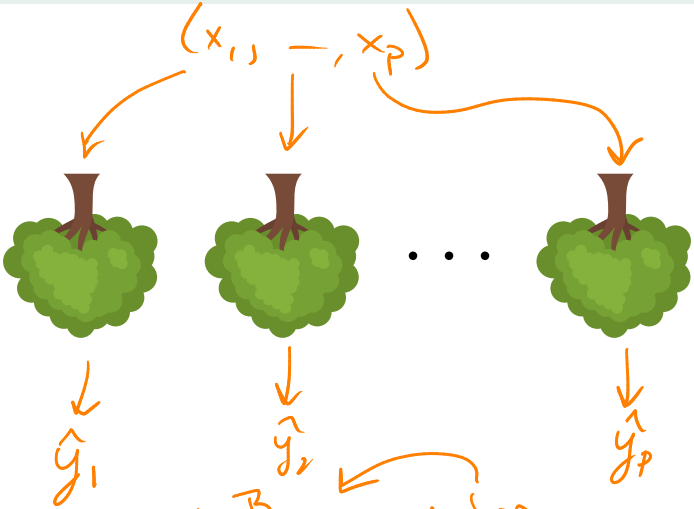
$$\hat{f}_{bag}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$$

or majority vote (classification)

Tree version



Prediction on new data point



Regression $\Rightarrow$ $\frac{1}{B}\sum_{b=1}^{B} \hat{y}_b$ prediction

Classification max'm vote

Bagging vs Bootstrap

Bootstrap

Get a subsample of data w/ replacement

↳ Better called "Bootstrap sample"

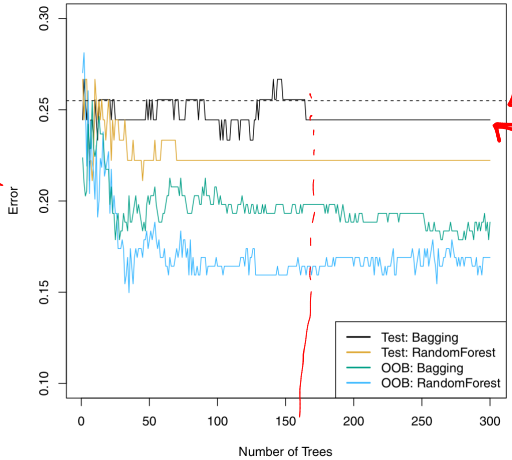
Bagging

Taking predicted values and returning the average

Better called "Bootstrap aggregation"

Bootstrapping / Bagging can be used in many models we just happen to be using trees

Example: Heart classification data

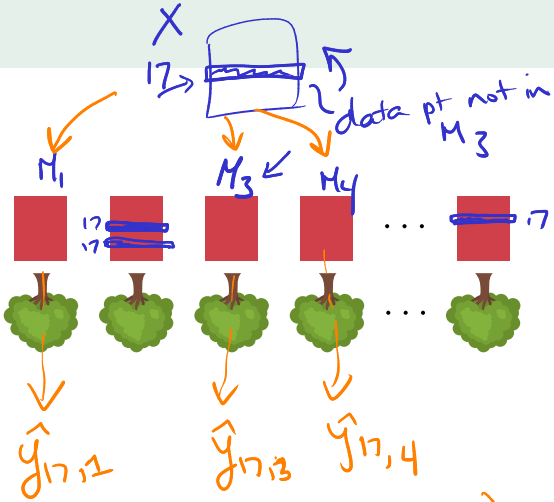


error from single decision tree

Test error from bagged version

Out of Bag Error Estimation

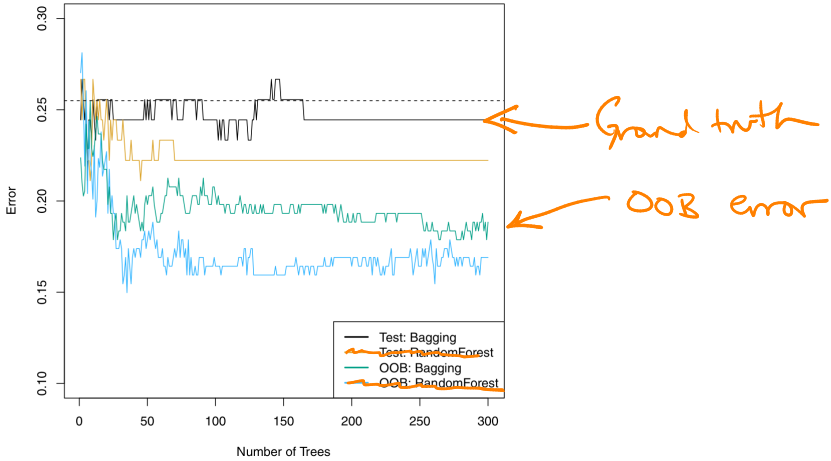
- On average, bootstrap sample uses about 2/3 of the data
- Remaining observations not used are called *out-of-bag* (OOB) observations
- For each observation, run through all the trees where it wasn't used for building
- Return the average (or majority vote) of those as test prediction



- Use RSS, or error %

$$\text{Test Prediction} = \frac{\left(\hat{y}_{17,1} + \hat{y}_{17,3} + \hat{y}_{17,4}\right)}{3}$$

Error using OOB



Section 3

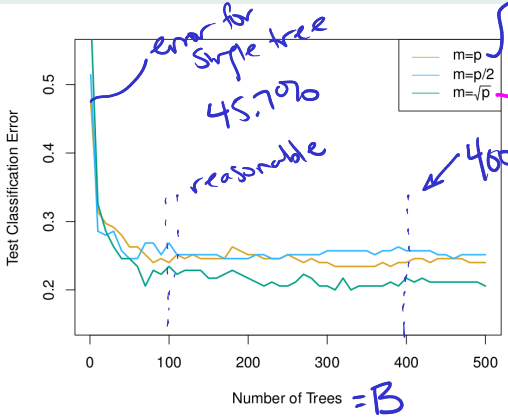
Random Forests

The idea

- Goal is to decorrelate the bagged trees:
  - If there is a strong predictor, the first split of most trees will be the same
  - Most or all trees will be highly correlated
  - Averaging highly correlated quantities doesn't decrease variance as much as uncorrelated

- The random forrest fix:
  - Each time a split is considered, only use a random subset of m the predictors ← p z
  - Fresh sample taken every time
  - Typically $m \approx \sqrt{p}$
  - On average, $(p - m)/p$ of splits won't consider strong predictor
  - $m = p$ gives back bagging

Example on gene expression



- Predict patient had one of 14 kinds of cancer or was healthy (15 levels)
- Use 500 genes here w/ most variance
- Random split of data to keep for testing

Section 4

Boosting

Boosting idea

Algorithm 8.2 *Boosting for Regression Trees*

1. Set $\hat{f}(x) = 0$ and $r_i = y_i$ for all i in the training set.
2. For $b = 1, 2, \ldots, B$, repeat:
   (a) Fit a tree $\hat{f}^b$ with d splits ($d+1$ terminal nodes) to the training data (X, r).
   (b) Update $\hat{f}$ by adding in a shrunken version of the new tree:
   $$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \tag{8.10}$$
   (c) Update the residuals,
   $$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i). \tag{8.11}$$
3. Output the boosted model,
   $$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x). \tag{8.12}$$

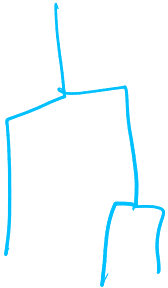
i

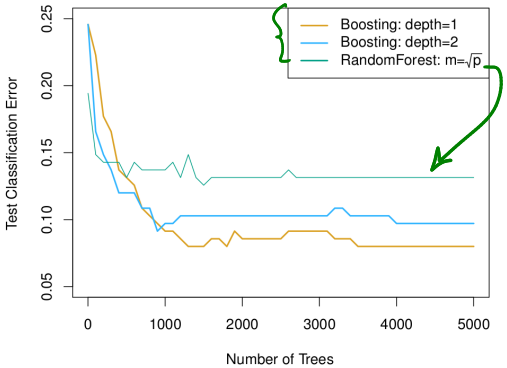
d = depth

prediction

- Create group of trees
- Instead of using bootstrap data, I'll use modified data
- Growing trees sequentially
- λ controls "speed" of learning
- Slow learning ← small λ ⇝ better prediction

Example




Boosting: depth=1
Boosting: depth=2
RandomForest: m=√p
Test Classification Error
0.05
0.10
0.15
0.20
0.25
0
1000
2000
3000
4000
5000
Number of Trees

Section 5

BART

BART

Bayesian Additive Regression tree

- Create trees in random manner (like bagging, random forests, and boosting)
- Try to capture signal not yet accounted for (like boosting)
- Change how trees are generated

Notation

- K: number of regression trees
- B: number of iterations to run the BART algorithm
- $\hat{f}_k^b(x)$: Prediction at x for kth regression tree for bth iteration
- $\hat{f}^b(x) = \sum_{k=1}^{K} \hat{f}_k^b$

BART Algorithm idea

- Start with single root node for each tree

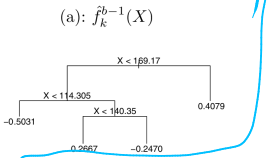
$$\hat{f}_k^1(x) = \frac{1}{nK}\sum_{i=1}^{n} y_i$$

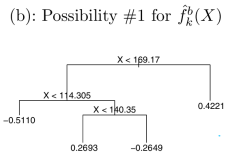
- Update one tree at a time. Update kth tree at bth iteration, get partial residual:

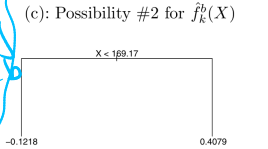
$$r_i = y_i - \sum_{k'<k} \hat{f}_{k'}^{b}(x_i) - \sum_{k'>k} \hat{f}_{k'}^{b-1}(x_i)$$

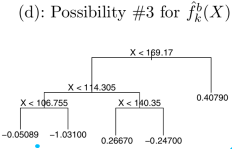
Measure of quality now = b

- Randomly choose pertubation to tree





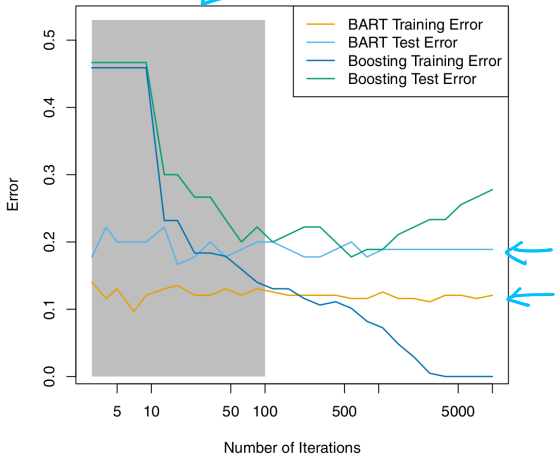




remove

Example

Burn in period



TL:DR

Bagging

Boosting

Random Forests

BART

Next time

Next time:

- Lab: Trees

Timing good

Check Boosting / BART Slides

Pretty sure I fell asleep making them

This week:

- Mon 3/28: 8.1
- Weds 3/30: 8.2
- Fri 4/1: Lab: 8.3
  - HW #8 Due Friday
- Next week: No class